

Speedrunning NanoGPT mit Muon

Report

Windisch, Mai 2026

Student

Jelle Schutter

Fach-Experte

Michael Graber

Modul

Optimierung für Machine Learning

Studiengang

Data Science & AI

Semester

FS26

Inhaltsverzeichnis

1	Muon Optimizer	1
2	NanoGPT Speedrunning	3
3	Speedrunning auf dem FHNW Calculon	5
3.1		5
	Abbildungsverzeichnis	6
	Tabellenverzeichnis	6
	Quellenverzeichnis	6

1 Muon Optimizer

Muon (*MomentUm Orthogonalized by Newton-Schulz*) ist ein von Keller Jordan et al. [1] entwickelter Optimizer für Hidden Layers in neuronalen Netzwerken. Die Grundidee: Statt die Aktualisierungen der Gewichte vom SGD-Momentum direkt anzuwenden, wendet Muon eine Orthogonalisierung an [1].

Algorithmus

Der Muon Algorithmus besteht aus den folgenden Schritten:

Require: Learning rate η , momentum μ

- 1: Initialize $B_0 \leftarrow 0$
- 2: **for** $t = 1, \dots$ **do**
- 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
- 4: $B_t \leftarrow \mu B_{t-1} + G_t$
- 5: $O_t \leftarrow \text{NewtonSchulz5}(B_t)$
- 6: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
- 7: **end for**
- 8: **return** θ_t

Abbildung 1.1: Der Muon-Algorithmus [1]

In jedem Schritt t wird zuerst der Gradient G_t berechnet (Zeile 3). Daraus wird wie bei SGD ein Momentum-Puffer $B_t = \mu B_{t-1} + G_t$ gebildet, der die früheren Gradienten mit dem Faktor μ gewichtet (Zeile 4).

Der entscheidende Schritt ist danach die Orthogonalisierung $O_t = \text{NewtonSchulz5}(B_t)$ (Zeile 5): B_t wird durch die approximierte Orthogonal-Matrix ersetzt [1]. Formal ist das die Operation $UV^\top$, wobei $B_t = USV^\top$ die Singulärwertzerlegung (SVD) von B_t ist.

Zum Schluss werden die Gewichte mit der Lernrate η aktualisiert: $\theta_t = \theta_{t-1} - \eta O_t$ (Zeile 6).

Newton-Schulz

Die Newton-Schulz-Iteration ist eines der Kernstücke Kernkonzepte von Muon, welche die Orthogonalisierung effizienter macht. Dies ist möglich durch die wiederholte Matrixmultiplikationen mit der folgenden Formel [1]:

$$X_{k+1} = a X_k + (b X_k X_k^\top + c (X_k X_k^\top)^2) X_k \quad (1.1)$$

mit ursprünglich den Koeffizienten $(a, b, c) = (3.4445, -4.7750, 2.0315)$. Nach typischerweise 5 Schritten konvergiert X_k gegen $UV^\top$. Gemäss Keller [1] konnte dies effizient und stabil in `bf16` durchgeführt werden, was schlussendlich ein Vorteil gegenüber anderen Orthogonalisierungsverfahren war.

In neueren Versionen werden die Koeffizienten a, b, c beim Speedrunning nicht mehr verwendet, stattdessen werden die Koeffizienten bei jedem Schritt neu berechnet um die Konvergenz weiter zu beschleunigen [2].

Warum Orthogonalisierung hilft

Die Update-Matrizen in Neuralen Netzwerken werden meist von wenigen starken Richtungen dominiert, während viele andere Richtungen nur sehr kleine Werte haben [1]. Ohne Gegenmassnahme folgt der Optimizer fast nur diesen wenigen starken Richtungen, und die vielen schwachen Richtungen werden kaum gelernt.

Genau hier setzt die Orthogonalisierung an: Sie bringt alle Richtungen auf die gleiche Stärke [1]. Dadurch erhalten auch die schwachen, aber durchaus auch nützlichen Richtungen einen gleich grossen Lernschritt, und das Modell nutzt die Information im Gradienten besser aus.

Rechenaufwand vs. Nutzen

Der Mehraufwand durch die Newton-Schulz-Iteration gegenüber dem Standard SGD-Momentum ohne Orthogonalisierung ist klein. Die zusätzlichen Matrixmultiplikationen können effizient auf der GPU durchgeführt werden [1]. Für den NanoGPT-Speedrun mit 768 Dimensionen und einer Batch-Size von 524288 ergaben Tests nur einen **0.7%** Overhead während die Anzahl der benötigten Iterationen signifikant reduziert wurde [1].

2 NanoGPT Speedrunning

Das Modded-NanoGPT Projekt möchte das Training von NanoGPT in so kurzer Zeit wie möglich erreichen. Die Zeit stoppt wenn der Validierungs-Loss von 3.28 erreicht wird [3]. Seit dem Start des Projekts ist die Trainingszeit von über 45 Minuten (Mai 2024) auf etwa 1,4 Minuten gesunken [3].

Optimierungen

Um diese Rekord-Zeiten zu erreichen, wurden viele Optimierungen am Modell, dem Training und der Hardware-Auslastung vorgenommen. Die wichtigsten davon werden im Folgenden vorgestellt.

Muon als zentraler Optimizer

Der grösste Gewinn kommt vom Wechsel von AdamW zu Muon für die Attention- und MLP-Gewichte im Transformer [3]. Embedding- und Output-Layer bleiben bei AdamW [1]. Seit der Einführung von Muon am 15. Oktober 2024 wurden alle Rekorde damit erzielt [1].

Architektur

Auch am Modell selbst wurde geschraubt. Mehrere bewährte Bausteine moderner Sprachmodelle ersetzen die ursprünglichen Komponenten von GPT-2:

- **RoPE** (Rotary Positional Embeddings) statt gelernter Positionskodierungen [3].
- **Value-Embeddings**: zusätzliche Embeddings, die in die Values der Attention-Schichten gemischt werden [3].
- **ReLU²-Aktivierungen** statt GELU in den Feed-Forward-Schichten [3].
- Keine Bias-Parameter [3].

Trainingseffizienz

Neben dem Modell wurde vor allem an der Hardware-Auslastung gefeilt, damit jede GPU-Sekunde möglichst viel Rechenarbeit erledigt:

- **FlashAttention** senkt den Speicher- und Rechenaufwand der Attention-Berechnung [3].
- **BF16 Mixed-Precision** und **torch.compile** nutzen die GPU besser aus [3].
- **Distributed Data Parallel (DDP)** für Multi-GPU-Training [3].

Feinabstimmung von Muon

Über den reinen Wechsel zu Muon hinaus wurde der Optimizer selbst weiter verbessert. Die folgenden vier Anpassungen brachten je rund ein bis zwei Prozent kürzere Trainingszeit [2].

NorMuon (adaptives Muon)

Adam gibt jedem Parameter eine eigene Lernrate, indem es den Gradienten durch seine geschätzte Schwankung teilt. Das dämpft verrauschte Gradienten und bremst in stark gekrümmten Bereichen der Loss-Landschaft [2]. Muon profitiert nicht von dieser Normalisierung pro Eintrag, weil sie die orthogonalisierte Struktur des Updates zerstören würde [2]. NorMuon ist ein Mittelweg: Es normalisiert nur entlang einer Achse und stellt danach die ursprüngliche Stärke des Updates wieder her. Zusammen mit angepasster Lernrate brachte das eine Zeitreduktion von rund 2 % [2].

Batch-Size-Scheduling

Kleine Batches sind token-effizienter (jeder Token liefert mehr Signal), grosse Batches sind schneller pro Token, da die GPU über die Batch-Dimension parallelisiert [2]. Muon verträgt grössere Batches als Adam, ohne Token zu verschwenden [2]. Praktisch heisst das: Die Batch-Grösse wird während des Trainings schrittweise erhöht. Anfangs lernt das Modell häufige Muster,

für die schon kleine Batches reichen; später folgen seltene Muster, die auch bei grossen Batches verrauscht bleiben. Das sparte rund 1,7 % Trainingzeit [2].

Polar Express (schnellere Orthogonalisierung)

Die Orthogonalisierung ist das Herzstück von Muon. Die Standard-Newton-Schulz-Iteration verwendet feste Koeffizienten und ignoriert den Zustand der jeweiligen Matrix [2]. Polar Express sucht stattdessen die optimalen Koeffizienten für jeden einzelnen Iterationsschritt und konvergiert dadurch schneller. Eine genauere Orthogonalisierung erlaubt zudem eine höhere Lernrate und stabileres Training, was einen neuen Rekord ergab [2].

Cautious Weight Decay

Weight Decay hält die Gewichte davon ab, zu gross zu werden. Beim kleinen Modded-NanoGPT schadete das Standardverfahren jedoch [2]. Cautious Weight Decay verkleinert nur die Parameter, die sich im Update ohnehin in diese Richtung bewegen, und arbeitet so nicht gegen den Optimizer. Das brachte rund 1,3 %, [2].

Verteilte Berechnung

Damit Muon auch auf mehreren GPUs schnell läuft, kommen mehrere Engineering-Tricks zum Einsatz [2]:

- Parameter gleicher Form werden gestapelt, damit die Orthogonalisierung gebündelt (vektoriert) abläuft. Attention-Gewichte werden dafür passend zu den MLP-Schichten zusammengefasst [2].
- Eigene Triton-Kernels nutzen die symmetrische Struktur der Newton-Schulz-Matrizen aus und halbieren so etwa die Rechenarbeit [2].
- Jede GPU erhält einen gleich grossen Teil der Gradienten. Die Kommunikation zwischen den GPUs läuft asynchron parallel zur Berechnung innerhalb der GPUs [2].

3 Speedrunning auf dem FHNW Calculon

Für die Experimente zum NanoGPT-Speedrun wurde die FHNW-Calculon-Clusterinfrastruktur genutzt, welches zwei Nodes mit je 4 H200-GPUs bietet. Alle Experimente wurden mit der Modded-NanoGPT Codebasis von Jordan Keller et al. [3] durchgeführt.

Setup und Konfiguration

Zuerst wurde versucht das offizielle Docker Image von Jordan Keller et al. [3] zu verwenden, in dem es zu einem Singularity Container umgewandelt wurde. Leider gab es dabei verschiedene Probleme, weshalb schlussendlich die Codebasis direkt auf dem Cluster installiert und ausgeführt wurde. Als einzige Änderung zur Originalcodebasis wurde das Package `keruels` auf Version 0.12 festgelegt, da die Hugging Face API in der aktuelleren Version nicht mit dem Projekt kompatibel war.

3.1

Leider konnte nur eine limitierte Anzahl von Experimenten durchgeführt werden, da die Cluster-Ressourcen stark gefragt sind. (Die warte Zeit für 4 GPUs war mehr als eine Woche.) Daher wurden nur die folgenden zwei Konfigurationen getestet werden:

- Training mit einer GPU (Baseline).
- Training mit zwei GPUs (Multi-GPU).

Die Rekordzeiten von etwa 1,4 Minuten wurden mit nur zwei H200-GPUs nicht erreicht, aber die Ergebnisse zeigen dennoch, dass mit zwei GPUs auch schon eine deutliche Beschleunigung möglich ist. Die folgende Tabelle fasst die gemessenen Trainingszeiten für verschiedene Konfigurationen zusammen:

Tabelle 3.1: Trainingszeiten des NanoGPT-Speedruns auf FHNW H200-GPUs

Konfiguration	Dauer (ms)	Dauer (min)
1 H200	630 917	$\approx 10,5$
2 H200	319 728	$\approx 5,3$

Der Übergang von einer auf zwei GPUs ergibt einen Speedup von $\approx 1,97\times$. Die Trainingszeit von etwa 5,3 Minuten mit zwei GPUs liegt zwar deutlich über den Rekordzeiten von 1,4 Minuten, zeigt aber dennoch eine signifikante Verbesserung gegenüber der Ein-GPU-Konfiguration. Es ist wahrscheinlich, dass weitere Optimierungen und die Nutzung aller vier GPUs zu noch schnelleren Trainingszeiten führen könnten.

Möglicherweise könnten auch die anderen 4 GPUs des Clusters noch genutzt werden, um die Zeiten weiter zu verbessern. Hierfür wäre es jedoch notwendig, die Codebasis entsprechend anzupassen, damit der Code auch auf mehreren Nodes laufen kann, was über die Möglichkeiten dieses Projekts hinausgeht und auch zu einem erheblichen Overhead führen könnte. Dennoch zeigt dieses Experiment, dass mit der richtigen Hardware und Optimierung auch auf einem Cluster wie dem FHNW Calculon deutliche Verbesserungen bei der Trainingszeit von Modellen wie NanoGPT möglich sind.

Abbildungsverzeichnis

1.1 Der Muon-Algorithmus [1]	1
--	---

Tabellenverzeichnis

3.1 Trainingszeiten des NanoGPT-Speedruns auf FHNW H200-GPUs	5
--	---

Quellenverzeichnis

- [1] K. Jordan u. a., *Muon: An optimizer for hidden layers in neural networks*, 2024. Adresse: <https://kellerjordan.github.io/posts/muon/>
- [2] S. Varun, *Muon in Modded NanoGPT*, 2025. Adresse: <https://varunneal.github.io/essays/muon>
- [3] K. Jordan u. a., *modded-nanogpt: Speedrunning the NanoGPT baseline*, 2024. Adresse: <https://github.com/KellerJordan/modded-nanogpt>